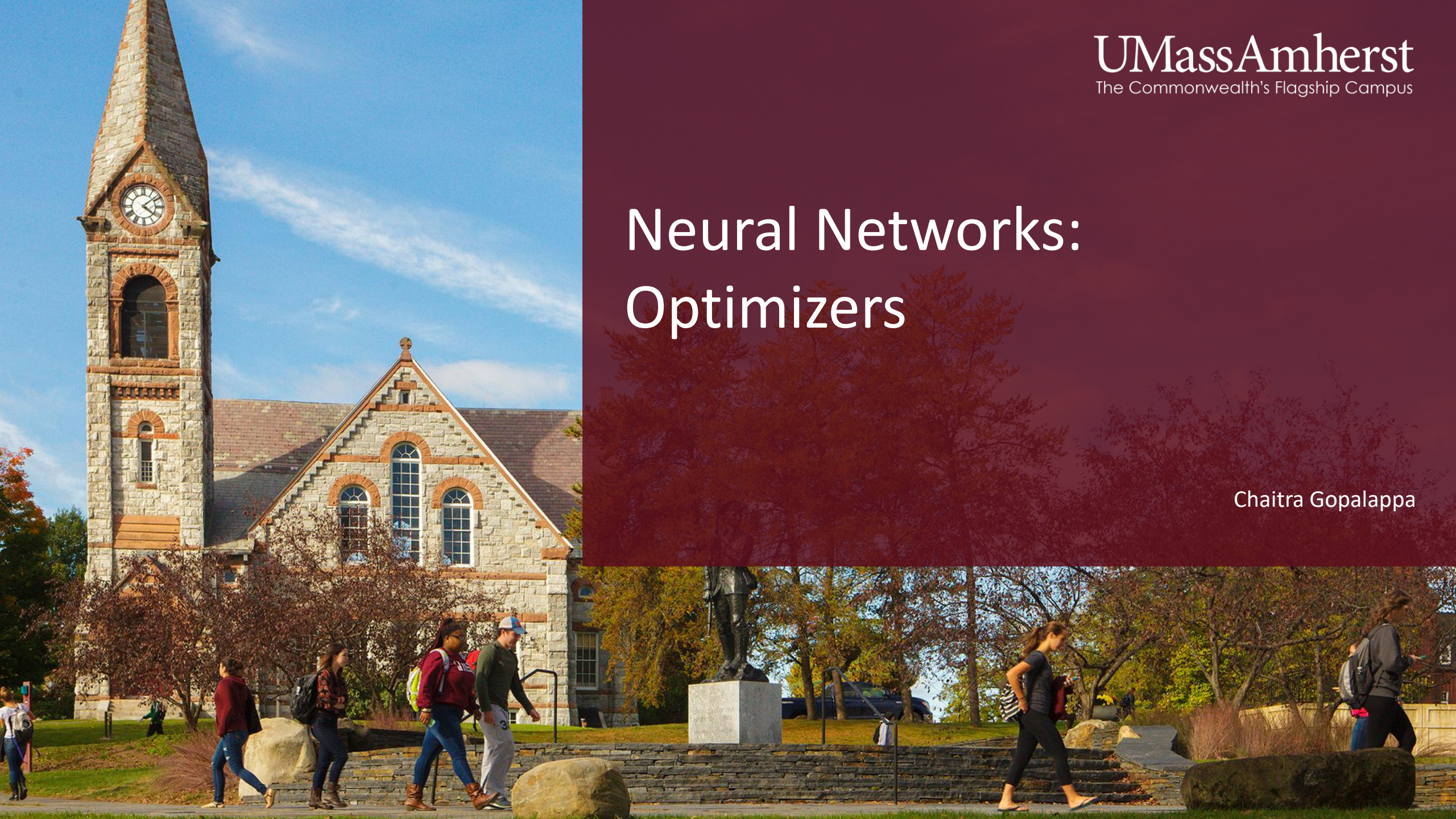


Neural Networks: Optimizers

Chaitra Gopalappa



Neural Network Optimizers

Source:

Chapter 8: Ian Goodfellow and Yoshua Bengio and Aaron Courville, Deep Learning

<https://www.deeplearningbook.org/contents/optimization.html>

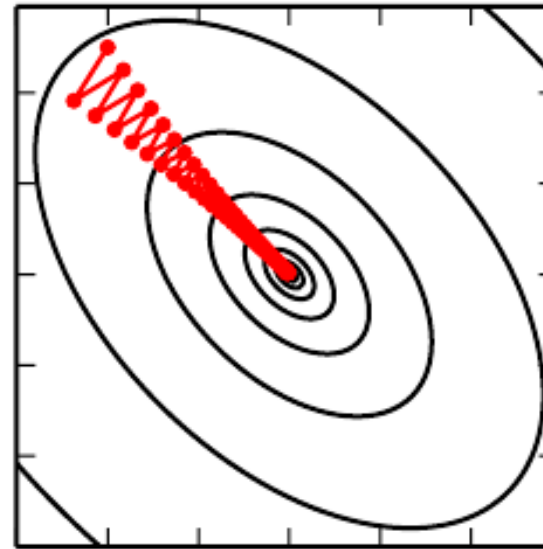
Line search algorithm

Transformation

$$\vec{x}_{m+1} \leftarrow \vec{x}_m + \mu_m \vec{p}_m$$

$\vec{p}_m$: search direction

μ_m : step size

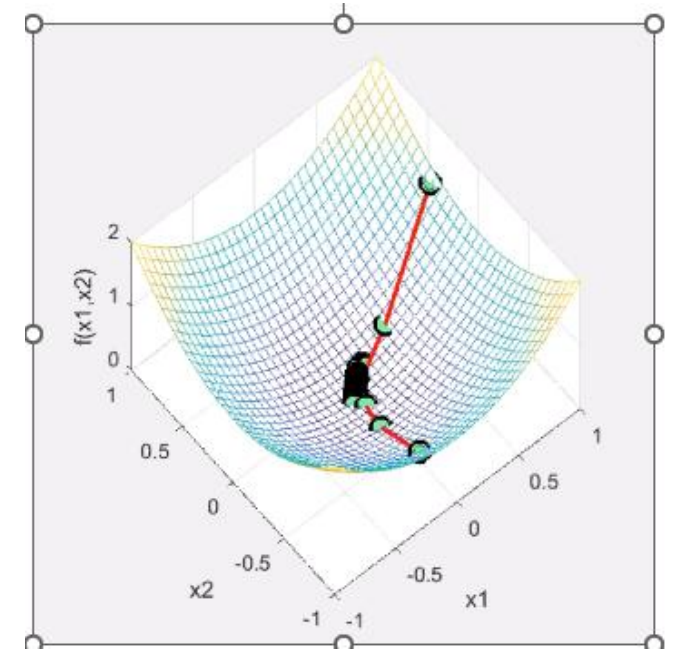


Stochastic gradient Descent (SDG) algorithm

Transformation

$$\vec{x}_{m+1} \leftarrow \vec{x}_m - \mu_m (\nabla f(\vec{x}_m))$$

$\vec{p}_m = -\nabla f(\vec{x}_m)$: search direction is the negative of the gradient

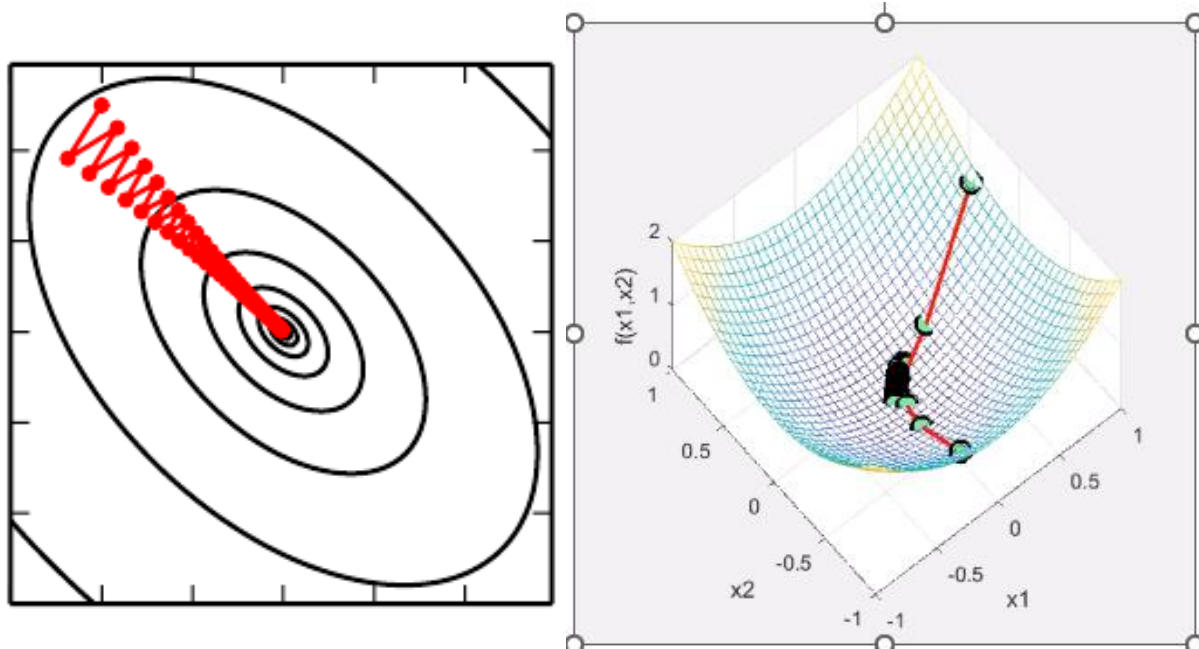


SDG in machine learning

Stochastic gradient Descent (SDG) algorithm

Transformation

$$\vec{x}_{m+1} \leftarrow \vec{x}_m - \mu_m (\nabla f(\vec{x}_m))$$



Algorithm 8.1 Stochastic gradient descent (SGD) update

Require: Learning rate schedule $\epsilon_1, \epsilon_2, \dots$

Require: Initial parameter θ

$k \leftarrow 1$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

 Apply update: $\theta \leftarrow \theta - \epsilon_k \hat{\mathbf{g}}$

$k \leftarrow k + 1$

end while

If $m = B$ (batch size/ full training dataset) it is general GD

If $m = 1$ it is incremental or online SGD

If $1 < m < B$ it is mini-batch SDG; typically $m = 2^n$ (some exponent of 2)

SDG in machine learning

Stochastic gradient Descent (SDG) algorithm

Transformation

$$\vec{x}_{m+1} \leftarrow \vec{x}_m - \mu_m (\nabla f(\vec{x}_m) + \text{noise})$$

Algorithm 8.1 Stochastic gradient descent (SGD) update

Require: Learning rate schedule $\epsilon_1, \epsilon_2, \dots$

Require: Initial parameter θ

$k \leftarrow 1$

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

Apply update: $\theta \leftarrow \theta - \epsilon_k \hat{\mathbf{g}}$

$k \leftarrow k + 1$

end while

Sufficient conditions for convergence

$$\sum_{k=1}^{\infty} \epsilon_k = \infty, \quad \text{and} \quad \sum_{k=1}^{\infty} \epsilon_k^2 < \infty.$$

Typical to **decay learning rate** linearly until some iteration τ , with $\alpha = \frac{k}{\tau}$. After iteration τ , leave ϵ constant.

$\epsilon_0, \epsilon_{\tau}, \tau$ are hyperparameters

$$\epsilon_k = (1 - \alpha)\epsilon_0 + \alpha\epsilon_{\tau}$$

Line search algorithm

Transformation

$$\vec{x}_{m+1} \leftarrow \vec{x}_m - \mu_m \vec{p}_m$$

$\vec{p}_m$: search direction

μ_m : step size

Neural Network Optimizers

- SDG
- SDG with momentum
- SDG with Nesterov

Gradient as
search direction

- AdaGrad
- RMSProp
- Adam

Adaptive learning
rate/ step size

- Newtons method
- Conjugate
gradients
- BFGS

Hessian as
search direction

Line Search
methods: identify
search direction
and step in that
direction

SDG with momentum

Weighted average of previous velocity (v) and current direction ($-g$); If ($-g$) consistently points in the same direction, the velocity builds up to go faster (take a larger step in that direction)

Algorithm 8.2 Stochastic gradient descent (SGD) with momentum

Require: Learning rate ϵ , momentum parameter α

Require: Initial parameter θ , initial velocity v

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient estimate: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$.

 Compute velocity update: $\mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \mathbf{g}$.

 Apply update: $\theta \leftarrow \theta + \mathbf{v}$.

end while

SDG with Nesterov momentum

With Nesterov momentum, the gradient is calculated at an “interim” step using the current momentum (step a little further using current momentum and calculate gradient at that new step to check if any correction is needed) and then update velocity (improves stability in learning). Addresses issues of being stuck in local minima

Algorithm 8.3 Stochastic gradient descent (SGD) with Nesterov momentum

Require: Learning rate ϵ , momentum parameter α

Require: Initial parameter θ , initial velocity v

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding labels $\mathbf{y}^{(i)}$.

 Apply interim update: $\tilde{\theta} \leftarrow \theta + \alpha v$.

 Compute gradient (at interim point): $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\tilde{\theta}} \sum_i L(f(\mathbf{x}^{(i)}; \tilde{\theta}), \mathbf{y}^{(i)})$.

 Compute velocity update: $\mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \mathbf{g}$.

 Apply update: $\theta \leftarrow \theta + \mathbf{v}$.

end while

Source: Chapter 8:

<https://www.deeplearningbook.org/contents/optimization.html>

Algorithms with Adaptive Learning Rates

Adapt learning rate specific to feature instead of using same for all features (divide the global learning rate by the square root of the sum of the squared historical gradients for that parameter.) (+delta to prevent division by zero)

Algorithm 8.4 The AdaGrad algorithm **Ada(ptive)Grad(ient)**

Require: Global learning rate ϵ

Require: Initial parameter θ

Require: Small constant δ , perhaps 10^{-7} , for numerical stability

Initialize gradient accumulation variable $\mathbf{r} = \mathbf{0}$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$.

 Accumulate squared gradient: $\mathbf{r} \leftarrow \mathbf{r} + \mathbf{g} \odot \mathbf{g}$.

 Compute update: $\Delta \theta \leftarrow -\frac{\epsilon}{\delta + \sqrt{\mathbf{r}}} \odot \mathbf{g}$. (Division and square root applied element-wise)

 Apply update: $\theta \leftarrow \theta + \Delta \theta$.

end while

Algorithms with Adaptive Learning Rates

Default decay rate
 $\rho \sim 0.9$ or 0.99
Exponentially decaying moving average –
dynamic adaptation of learning rate.
Can address issues of vanishing or
exploding gradient

With Nesterov momentum

Algorithm 8.5 The RMSProp algorithm **Root Mean Square propagation**

Require: Global learning rate ϵ , decay rate ρ
Require: Initial parameter θ
Require: Small constant δ , usually 10^{-6} , used to stabilize division by small numbers

Initialize accumulation variables $\mathbf{r} = \mathbf{0}$
while stopping criterion not met **do**
 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.
 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$.
 Accumulate squared gradient: $\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g} \odot \mathbf{g}$.
 Compute parameter update: $\Delta \theta = -\frac{\epsilon}{\sqrt{\delta + \mathbf{r}}} \odot \mathbf{g}$. ($\frac{1}{\sqrt{\delta + \mathbf{r}}}$ applied element-wise)

 Apply update: $\theta \leftarrow \theta + \Delta \theta$.
end while

Algorithm 8.6 RMSProp algorithm with Nesterov momentum

Require: Global learning rate ϵ , decay rate ρ , momentum coefficient α
Require: Initial parameter θ , initial velocity $\mathbf{v}$

Initialize accumulation variable $\mathbf{r} = \mathbf{0}$
while stopping criterion not met **do**
 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.
 Compute interim update: $\tilde{\theta} \leftarrow \theta + \alpha \mathbf{v}$.
 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\tilde{\theta}} \sum_i L(f(\mathbf{x}^{(i)}; \tilde{\theta}), \mathbf{y}^{(i)})$.
 Accumulate gradient: $\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g} \odot \mathbf{g}$.
 Compute velocity update: $\mathbf{v} \leftarrow \alpha \mathbf{v} - \frac{\epsilon}{\sqrt{\mathbf{r}}} \odot \mathbf{g}$. ($\frac{1}{\sqrt{\mathbf{r}}}$ applied element-wise)
 Apply update: $\theta \leftarrow \theta + \mathbf{v}$.
end while

Source: Chapter 8:

<https://www.deeplearningbook.org/contents/optimization.html>

Algorithms with Adaptive Learning Rates

Adaptive Moment estimation

Exponentially decaying moving average of second moment (squared of gradient) similar to RMSprop. And exponentially decaying moving average of first moment (mean of gradient). The first moment is used for the search direction (notice it is similar to velocity in SGD momentum and plays similar role). The second moment is used for the learning rate (step size)

Initially it can be biased towards zero as s and r are initialized to zero. So correct for that bias

Algorithm 8.7 The Adam algorithm

Require: Step size ϵ (Suggested default: 0.001)

Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0, 1)$. (Suggested defaults: 0.9 and 0.999 respectively)

Require: Small constant δ used for numerical stabilization (Suggested default: 10^{-8})

Require: Initial parameters θ

Initialize 1st and 2nd moment variables $\mathbf{s} = \mathbf{0}$, $\mathbf{r} = \mathbf{0}$

Initialize time step $t = 0$

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

$t \leftarrow t + 1$

Update biased first moment estimate: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g}$

Update biased second moment estimate: $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g} \odot \mathbf{g}$

Correct bias in first moment: $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}$

Correct bias in second moment: $\hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$

Compute update: $\Delta \theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$ (operations applied element-wise)

Apply update: $\theta \leftarrow \theta + \Delta \theta$

end while

Source: Chapter 8:

<https://www.deeplearningbook.org/contents/optimization.html>

SGD

SDG function in Pytorch

```
class torch.optim.SGD(params, lr=0.001, momentum=0, dampening=0, weight_decay=0, nesterov=False,  
*, maximize=False, foreach=None, differentiable=False, fused=None)
```

[\[source\]](#)

Implements stochastic gradient descent (optionally with momentum).

input : γ (lr), θ_0 (params), $f(\theta)$ (objective), λ (weight decay),
 μ (momentum), τ (dampening), *nesterov*, *maximize*

```
for  $t = 1$  to ... do  
  if maximize :  
     $g_t \leftarrow -\nabla_{\theta} f_t(\theta_{t-1})$   
  else  
     $g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$   
  if  $\lambda \neq 0$   
     $g_t \leftarrow g_t + \lambda \theta_{t-1}$   
  if  $\mu \neq 0$   
    if  $t > 1$   
       $\mathbf{b}_t \leftarrow \mu \mathbf{b}_{t-1} + (1 - \tau) g_t$   
    else  
       $\mathbf{b}_t \leftarrow g_t$   
    if nesterov  
       $g_t \leftarrow g_t + \mu \mathbf{b}_t$   
    else  
       $g_t \leftarrow \mathbf{b}_t$   
   $\theta_t \leftarrow \theta_{t-1} - \gamma g_t$   
  
return  $\theta_t$ 
```

- <https://pytorch.org/docs/stable/generated/torch.optim.SGD.html#torch.optim.SGD>
- Bengio, Practical recommendations for gradient-training of deep architectures, 2012, <https://arxiv.org/abs/1206.5533>

Pytorch Adam optimizer

Adam

```
class torch.optim.Adam(params, lr=0.001, betas=(0.9, 0.999), eps=1e-08, weight_decay=0,
amsgrad=False, *, foreach=None, maximize=False, capturable=False, differentiable=False,
fused=None, decoupled_weight_decay=False)
```

[\[source \]](#)

Implements Adam algorithm.

input : γ (lr), β_1, β_2 (betas), θ_0 (params), $f(\theta)$ (objective)
 λ (weight decay), *amsgrad*, *maximize*, ϵ (epsilon)

initialize : $m_0 \leftarrow 0$ (first moment), $v_0 \leftarrow 0$ (second moment), $v_0^{max} \leftarrow 0$

for $t = 1$ **to** ... **do**

if *maximize* :

$g_t \leftarrow -\nabla_{\theta} f_t(\theta_{t-1})$

else

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$

if $\lambda \neq 0$

$g_t \leftarrow g_t + \lambda \theta_{t-1}$

$m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$

$v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$

$\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$

if *amsgrad*

$v_t^{max} \leftarrow \max(v_{t-1}^{max}, v_t)$

$\widehat{v}_t \leftarrow v_t^{max} / (1 - \beta_2^t)$

else

$\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$

$\theta_t \leftarrow \theta_{t-1} - \gamma \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$

return θ_t

More variants added over time

- Best way to keep up is to look at NN libraries (Keras, Pytorch, Tensorflow) on available options
 - <https://keras.io/api/optimizers/>
 - <https://pytorch.org/docs/stable/optim.html>
 - https://www.tensorflow.org/api_docs/python/tf/keras/optimizers

UMassAmherst
The Commonwealth's Flagship Campus